Module 3: Understanding Data Formats

© 2016 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Importance of a Data Format

```
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet1       10.0.2.15       YES DHCP   up                    up
GigabitEthernet2       172.16.0.2      YES manual up                    up
GigabitEthernet3       172.17.0.1      YES manual up                    up
```


```
{
    "ietf-interfaces:interfaces": {
        "interface": [
            {
                "name": "GigabitEthernet2",
                "description": "Wide Area Network",
                "type": "iana-if-type:ethernetCsmacd",
                "enabled": true,
                "ietf-ip:ipv4": {
                    "address": [
                        {
                            "ip": "172.16.0.2",
                            "netmask": "255.255.255.0"
                        }
                    ]
                }
            },
            {
                "name": "GigabitEthernet3",
                "description": "Local Area Network",
                "type": "iana-if-type:ethernetCsmacd",
                "enabled": true,
                "ietf-ip:ipv4": {
                    "address": [
                        {
                            "ip": "172.17.0.1",
                            "netmask": "255.255.255.0"
                        }
                    ]
                }
            }
        ]
    }
}
```


s affiliates. All rights reserved. Cisco Confidential

The Data Formats Concept

- Data formats are simply a way to store and exchange data in a structured format.
- These are some common data formats that are used in many applications including network automation and programmability:
  - JavaScript Object Notation (JSON)
  - eXtensible Markup Language (XML)
  - YAML Ain’t Markup Language (YAML)
- The data format that is selected will depend on the format that is used by the application, tool, or script that you are using. Many systems will be able to support more than one data format, which allows the user to choose their preferred one.

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Common Data Formats in Programming

JSON

```
{
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet2",
        "description": "Wide Area Network”,
        "enabled": true,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "172.16.0.2",
                    "netmask": "255.255.255.0"
                }
            ]
        }
    }
}
```

XML

```
<?xml version="1.0" encoding="UTF-8" ?>
<interface xmlns="ietf-interfaces">
  <name>GigabitEthernet2</name>
  <description>Wide Area Network</description>
  <enabled>true</enabled>
  <ipv4>
    <address>
      <ip>172.16.0.2</ip>
      <netmask>255.255.255.0</netmask>
    </address>
  </ipv4>
</interface>
```

YAML

```
---
ietf-interfaces:interface:
  name: GigabitEthernet2
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
    - ip: 172.16.0.2
      netmask: 255.255.255.0
```

Understanding Data Formats

XML

- Extensible Markup Language (XML) is a generic methodology for wrapping textual data in symmetrical tags to indicate semantics.
- It is a derivative of Structured, Generalized Markup Language (SGML), and also the parent of HyperText Markup Language (HTML). XML filenames typically end in ".xml".
- XML documents are composed of **tags** and **attributes** that describe the structure and content of the data. **Tags** are used to define **elements** in the document, while **attributes** provide additional information about those elements.
- Elements can contain markup, including other elements, which are called "child elements".
- The largest, top-level element is called the **root**, which contains all other elements.

An Example XML Document

prologue = tag but start with '?' its optional, begging of file
identify virtual of XML document

comment = idea what is use for, what content xml gonna be, -- .....---

Root = vms = tag name

inside root element
- virtual machine ID
- virtual machine type

Prologue →
Comment →
Root Element →
Child Element →

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Instance list -->
<vms>
  <vm>
    <vmid>0101af9811012</vmid>
    <type>t1.nano</type>
  </vm>
  <vm>
    <vmid>0102bg8908023</vmid>
    <type>t1.micro</type>
  </vm>
</vms>
```

© 2020

Understanding Data Formats

XML (Contd.)

- **XML Document Body:** Except the first two lines of a XML document, the remainder of the document is considered as the body.
- **User-Defined Tag Names:** XML tag names are user-defined. If you are composing XML for your own application, pick tag names that clearly express the meaning of data elements, their relationships, and hierarchy.
- **Special Character Encoding:** Data is conveyed in XML as readable text.
  - To avoid having to individually find and convert special characters, it is possible to incorporate entire raw character strings in XML files by surrounding them with so-called **CDATA** blocks.

```
<hungarian_sentence><![CDATA[Minden személynek joga van a neveléshez.]]></hungarian_sentence>
```

CDATA = XML file

XML can have many XML distribute = describe XML file
-XML ID
- XML type
XML distribute have one is unique, there are their name

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

XML (Contd.)

- **XML Attributes:** XML lets you embed attributes within tags to convey additional information.
  - Attribute values must always be included in single or double quotes.
  - An element may have multiple attributes, but only one attribute for each specific name.
  - If an element has no content, you can use a shorthand notation in which you put the slash inside the open tag, rather than including a closing tag.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Instance list -->
<vms>
  <vm vmid="0101af9811012" type="t1.nano" />
  <vm vmid="0102bg8908023" type="t1.micro"/>
</vms>
```

have open tag

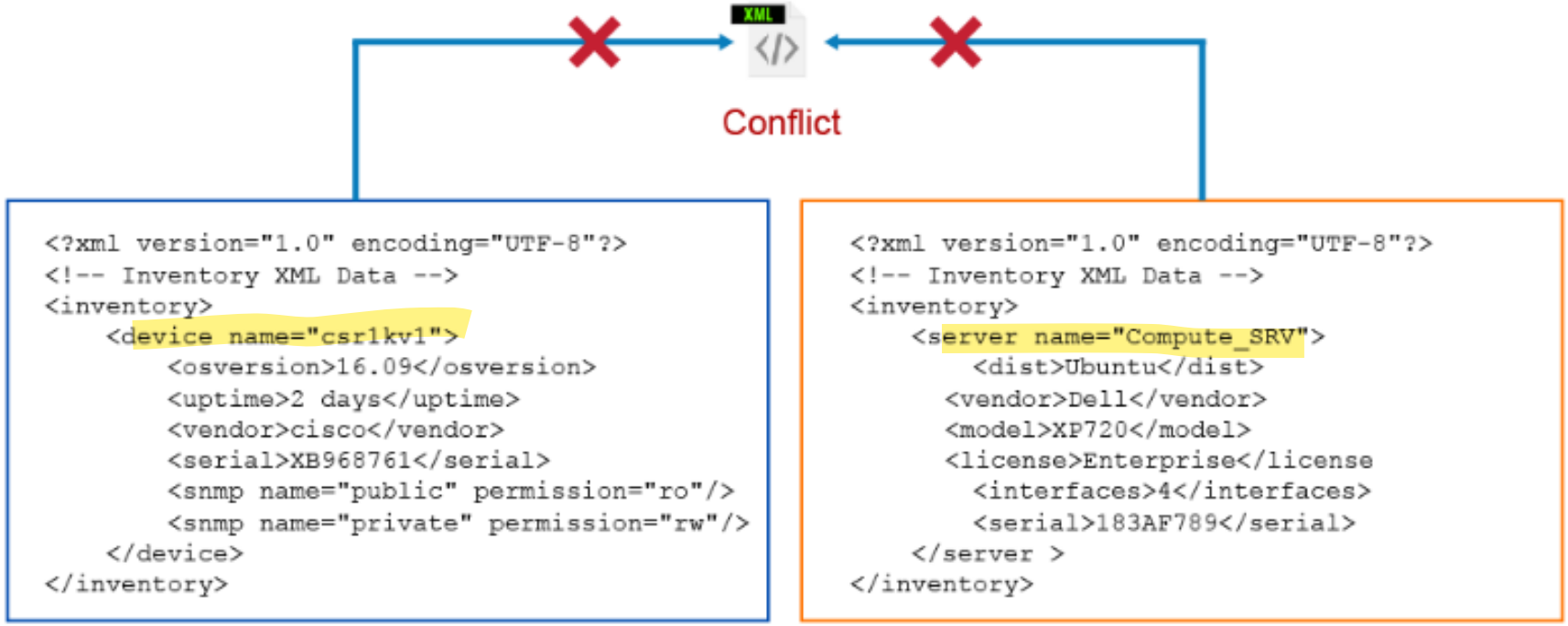
and close tag

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats

XML (Contd.)

- **XML Namespaces:**
- XML has the flexibility to name the tags in any manner you want. This flexibility creates a challenge.



if we have to merge different file then gonna create problem, so we create name space, so if you create file you have to follow the rule

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats

XML (Contd.)

- **XML Namespaces:** follow standard
  - Namespaces serve as unique XML identifiers. XML namespace is a mechanism used in XML to avoid element name conflicts between different XML vocabularies or languages. It allows elements and attributes in an XML document to be associated with a unique namespace identifier, which helps to distinguish them from similar elements and attributes in other namespaces.
  - Namespaces are defined by the IETF and other internet authorities, organizations, and other entities, and their schemas are typically hosted as public documents on the web.
  - The namespace is defined using a Uniform Resource Identifier (URI) or Uniform Resource Name (URN) that identifies the namespace. The namespace declaration is typically included as an attribute on an XML element, using the "**xmlns**" attribute.

name space

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <kill-session>
    <session-id>4</session-id>
  </kill-session>
</rpc>
```

XML (Contd.)

- **Interpreting XML**
  - In the XML example, the structure is represented as a list or one-dimensional array (called 'instances') of objects (each identified as an 'instance' by bracketing tags). Each instance object contains two key-value pairs denoting a unique instance ID and VM server type.
  - A semantically-equivalent Python data structure might be declared as shown below:

```
vms [
  {
    {"vmid": "0101af9811012"},
    {"type": "t1.nano"}
  },
  {
    {"vmid": "0102bg8908023"},
    {"type": "t1.micro"}
  }
]
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats

JSON

- JSON, or JavaScript Object Notation, is a data format derived from the way complex object literals are written in JavaScript.
- JSON filenames typically end in "**.json**."
- JSON is best known for the curly bracket ({}) syntax because JSON objects always **begin and end with a curly bracket**.
- JSON data is structured as a collection of name-value pairs, where the names (also known as keys) are strings and the values can be strings, numbers, arrays, objects, booleans, or null.
- JSON data is widely used in web development and APIs (Application Programming Interfaces) because it is easy to parse and generate using most programming languages.

j.son similar to dictionary

can be different type of data

```
{
  "edit-config":
  {
    "default-operation": "merge",
    "test-operation": "set",
    "some-integers": [2,3,5,7,9],
    "a-boolean": true,
    "more-numbers": [2.25E+2,-1.0735],
  }
}
```

string

```
{
    "inventory": {
        "device": [
            {
                "name": "csr1kv1",
                "version": "16.09",
                "vendor": "cisco",
                "uptime": "2 days",
                "serial": "XB96861",
                "snmp": [
                    {"name": "public", "permission": "ro"},
                    {"name": "private", "permission": "rw"},
                ],
            }
        ]
    }
}
```

Object Starts

Object Ends

JSON Syntax Rules

These are some of the characteristics of JSON:

- It uses a hierarchical structure and contains nested values.
- It uses braces { } to hold objects and square brackets [ ] hold arrays.
- Its data is written as key/value pairs.

With JSON, the data known as an **object** is one or more key/value pairs enclosed in braces { }. The syntax for a JSON object includes:

- Keys must be strings within double quotation marks " ".
- Values must be a valid JSON data type (string, number, array, Boolean, null, or another object).
- Keys and values are separated by a colon.
- Multiple key/value pairs within an object are separated by commas.
- White space is not significant.

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats

JSON (Contd.)

- **JSON Basic Data Types:** JSON basic data types include numbers, strings, Booleans, or nulls.
- **JSON Objects:** As in JavaScript, individual objects in JSON comprise of key/value pairs, which may be surrounded by braces, individually. {"keyname": "value"}
- **JSON Maps and Lists:** Objects may also contain multiple key/value pairs, separated by commas, creating structures equivalent to complex JavaScript objects, or Python dictionaries. In this case, each individual key/value pair does not need its own set of brackets, but the entire object does.
- **No Comments in JSON:** Unlike XML and YAML, JSON does not support any kind of standard method for including unparsed comments in code.
- **Whitespace Insignificant:** Whitespace in JSON is not significant,

  and files can be indented using tabs or spaces as preferred,

  or not at all.

```
{
  "edit-config":
  {
    "default-operation": "merge",
    "test-operation": "set",
    "some-integers": [2,3,5,7,9],
    "a-boolean": true,
    "more-numbers": [2.25E+2,-1.0735],
  }
}
```

show map of list each of them have their own 'Keyname'

multiple objects inside and have comma to saperate

have " " to identify "keyname" and "value"

© 2020 Cis

Understanding Data Formats

YAML

- YAML Ain't Markup Language (YAML) is a superset of JSON designed for even easier human readability.
- As a superset of JSON, YAML parsers can generally parse JSON documents (but not vice-versa).

YAML can generate some part of JSON

- Hence, YAML is better than JSON at certain tasks, including the ability to embed JSON directly (including quotes) in YAML files.
- YAML data consists of key-value pairs, lists, and nested structures. The data is represented using indentation and dashes to indicate the hierarchy and structure of the data.
- YAML data is widely used in configuration files for software applications, particularly in the context of containerization and orchestration platforms such as Docker and Kubernetes. Many popular programming languages, such as Python, Java, and Ruby, provide libraries for working with YAML data.

```yaml
---
edit-config:
  a-boolean: true
  default-operation: merge
  more-numbers:
  - 225.0
  - -1.0735
  some-integers:
  - 2
  - 3
  - 5
  - 7
  - 9
  test-operation: set
...
```

Understanding Data Formats

YAML (Contd.)

OPEN

- **YAML File Structure:** YAML files conventionally open with three dashes (--- alone on a line) which is called a document separator, and end with three dots (... likewise).

CLOSE

- **YAML Data Types:** YAML basic data types include numbers, strings, Booleans, or nulls.
- **Basic Objects:** In YAML, basic data types are equated to keys.
- **YAML Indentation and File Structure:** YAML indicates its hierarchy using indentation.

Treat as string

```
my_integer: 2
my_float: 2.1
my_exponent: 2e+5
'my_complex:key' : "my quoted string value\n"
0.2 : "can you believe that's a key?"
my_boolean: true
my_null: null # might be interpreted as empty string, otherwise
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats YAML (Contd.)

- **Maps and Lists:** YAML easily represents more complex data types, such as maps containing multiple key/value pairs and ordered lists.
  - Maps are generally expressed over multiple lines, beginning with a label key and a colon, followed by members, indented on subsequent lines:

```
mymap:
  myfirstkey: 5
  mysecondkey: The quick brown fox
```

  - Lists (arrays) are represented in a similar way, but with optionally-indented members preceded by a single dash and space:

```
mylist:
  - 1
  - 2
  - 3
```

  - Maps and lists can also be represented in a so-called "flow syntax," which looks very much like JavaScript or Python:

```
mymap: { myfirstkey: 5, mysecondkey: The quick brown fox}
mylist: [1, 2, 3]
```

Understanding Data Formats

YAML (Contd.)

- **Long Strings:** They are represented using a 'folding' syntax, where linebreaks are presumed to be replaced by spaces when the file is parsed/consumed, or in a non-folding syntax.

shows gonna be long line together

```
mylongstring: >
  This is my long string
  which will end up with no linebreaks in it
myotherlongstring: |
  This is my other long string
  which will end up with linebreaks as in the original
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

YAML (Contd.)

- **Comments:** Comments in YAML can be inserted anywhere except in a long string literal and are preceded by the hash sign and a space.

```
# this is a comment
```

- **More YAML Features:** YAML has many more features, most often encountered when using it in the context of specific languages, like Python, or when converting to JSON or other formats. For example, YAML 1.2 supports schemas and tags, which can be used to disambiguate interpretation of values.

  For example, to force a number to be interpreted as a string, you could use the !!str string, which is part of the YAML "Failsafe" schema:

```
mynumericstring: !!str 0.1415
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Understanding Data Formats

Parsing and Serializing

Parsing = you understand and able to read the file

- Parsing means analyzing a message, breaking it into its component parts, and understanding their purposes in context.
- Serializing is roughly the opposite of parsing(data stored in Python dictionaries → output as equivalent JSON, YAML or XML).

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Work with XML File

- XML is an inherently hierarchical data format, and the most natural way to represent it is with a tree. The xml.etree.ElementTree module is used to parse and create XML data. ET has two classes for this purpose - ElementTree represents the whole XML document as a tree, and Element represents a single node in this tree. Interactions with the whole document (reading and writing to/from files) are usually done on the ElementTree level. Interactions with a single XML element and its sub-elements are done on the Element level.

xml_read.py > ...

```python
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
root = tree.getroot()
print(root.tag)
print(root.attrib)
for child in root:
    print(child.tag,child.attrib)
```

.parse = read file

Output

command that gonna read this file

```
data
{}
country {'name': 'Liechtenstein'}
country {'name': 'Singapore'}
country {'name': 'Panama'}
```

root.tag

no data attributes

country_data.xml

```xml
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank>1</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank>4</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank>68</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

open ROOT

shell tag

in data have root elements

close

cisco

ts reserved. Cisco Confidential

Work with XML File-Finding interesting elements

- It is helpful to know all the elements in the entire tree. One useful function for doing that is **root.iter().** You can put this function into a "for" loop and it will iterate over the entire tree.

- **Element.findall()** finds only elements with a tag which are direct children of the current element. **Element.find()** finds the first child with a particular tag, and **Element.text** accesses the element's text content. **Element.get()** accesses the element's attributes:

```
for neighbor in root.iter('neighbor'):
    print(neighbor.attrib)
```

```
{'name': 'Austria', 'direction': 'E'}
{'name': 'Switzerland', 'direction': 'W'}
{'name': 'Malaysia', 'direction': 'N'}
{'name': 'Costa Rica', 'direction': 'W'}
{'name': 'Colombia', 'direction': 'E'}
```

```
for country in root.findall('country'):
    rank=country.find('rank').text
    name=country.get('name')
    print(name,rank)
```

```
Liechtenstein 1
Singapore 4
Panama 68
```

country_data.xml

```xml
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank>1</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank>4</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank>68</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

Convert XML to JSON

- an open-source module named **xmltodict** will be used to convert XML to JSON.
- The first step is to import the necessary modules:

```
>>> import xmltodict
>>> import json
```

- Then the file contents need to be read and parsed with the xmltodict parse method. The returned data will be assigned to an XML variable.

```
>>> with open('inventory.xml') as xml_file:
...     xml = xmltodict.parse(xml_file.read())
```

- The XML variable contains an object that can be converted to JSON using the **json.dumps()** method as follows:

```
>>> print(json.dumps(xml, indent=4))
```

- The XML attributes will be converted with @ at the beginning of the attribute's name, to get rid of:

```
>>> with open('inventory.xml') as xml_file:
...     xml = xmltodict.parse(xml_file.read(), attr_prefix='')
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Work with JSON Files

- Python comes with a built-in package called **json** for encoding and decoding JSON data.

```python
import json
```

- **json.load:** Reading **JSON file** and returns **Python objects(dictionary)**
- **json.loads:** reading **string in JSON format** and returns **Python objects(dictionary)**

sw_templates.json

```
{} sw_templates.json > ...
1  {
2    "access": [
3      "switchport mode access",
4      "switchport access vlan",
5      "switchport nonegotiate",
6      "spanning-tree portfast",
7      "spanning-tree bpduguard enable"
8    ],
9    "trunk": [
10     "switchport trunk encapsulation dot1q",
11     "switchport mode trunk",
12     "switchport trunk native vlan 999",
13     "switchport trunk allowed vlan"
14   ]
15 }
```

```
json_read_load.py > ...
1  import json
2
3  with open('sw_templates.json') as f:
4      templates = json.load(f)
5
6  print(templates)
7
8  for section, commands in templates.items():
9      print(section)
10     print('\n'.join(commands))
```

as ..... = no need to close the file

go to the key in dictionary

join with new line

```
access
switchport mode access
switchport access vlan
switchport nonegotiate
spanning-tree portfast
spanning-tree bpduguard enable
trunk
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk native vlan 999
switchport trunk allowed vlan
```

r its affiliates. All rights reserved. Cisco Confidential

Work with JSON Files

- **json.loads**: Reading JSON **string** to Python object

```python
import json

with open('sw_templates.json') as f:
    file_content = f.read()
    templates = json.loads(file_content)

print(templates)

for section, commands in templates.items():
    print(section)
    print('\n'.join(commands))
```

```
{'access': ['switchport mode access',
portfast', 'spanning-tree bpduguard en
rt mode trunk', 'switchport trunk nati
access
switchport mode access
switchport access vlan
switchport nonegotiate
spanning-tree portfast
spanning-tree bpduguard enable
trunk
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk native vlan 999
switchport trunk allowed vlan
```

```python
import json
json_string = """
{
    "researcher": {
        "name": "Ford Prefect",
        "species": "Betelgeusian",
        "relatives": [
            {
                "name": "Zaphod Beeblebrox",
                "species": "Betelgeusian"
            }
        ]
    }
}
"""
data = json.loads(json_string)

print(type(data))
print(data)
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Work with JSON Files

- **json.dump** - method writes Python object to file in JSON format

writeJson_dump.py > ...

```python
import json

trunk_template = [
    'switchport trunk encapsulation dot1q', 'switchport mode trunk',
    'switchport trunk native vlan 999', 'switchport trunk allowed vlan'
]

access_template = [
    'switchport mode access', 'switchport access vlan',
    'switchport nonegotiate', 'spanning-tree portfast',
    'spanning-tree bpduguard enable'
]

to_json = {'trunk': trunk_template, 'access': access_template}

with open('dump_sw_templates.json', 'w') as f:
    json.dump(to_json, f)

with open('dump_sw_templates.json') as f:
    print(f.read())
```

key

lists

value

json.dump return object into string

Work with JSON Files

- **json.dumps** - method returns **string** in JSON format

writeJson_dumps.py > ...

```python
import json

trunk_template = [
    'switchport trunk encapsulation dot1q', 'switchport mode trunk',
    'switchport trunk native vlan 999', 'switchport trunk allowed vlan'
]

access_template = [
    'switchport mode access', 'switchport access vlan',
    'switchport nonegotiate', 'spanning-tree portfast',
    'spanning-tree bpduguard enable'
]

to_json = {'trunk': trunk_template, 'access': access_template}

with open('dumps_sw_templates.json', 'w') as f:
    f.write(json.dumps(to_json))

with open('dumps_sw_templates.json') as f:
    print(f.read())
```

Additional Parameters of Write Methods

- Methods dump and dumps can pass additional parameters to manage the output format.

```python
import json

trunk_template = [
    'switchport trunk encapsulation dot1q', 'switchport mode trunk',
    'switchport trunk native vlan 999', 'switchport trunk allowed vlan'
]

access_template = [
    'switchport mode access', 'switchport access vlan',
    'switchport nonegotiate', 'spanning-tree portfast',
    'spanning-tree bpduguard enable'
]

to_json = {'trunk': trunk_template, 'access': access_template}

with open('sw_templates.json', 'w') as f:
    json.dump(to_json, f, sort_keys=True, indent=2)

with open('sw_templates.json') as f:
    print(f.read())
```

```
{
  "access": [
    "switchport mode access",
    "switchport access vlan",
    "switchport nonegotiate",
    "spanning-tree portfast",
    "spanning-tree bpduguard enable"
  ],
  "trunk": [
    "switchport trunk encapsulation dot1q",
    "switchport mode trunk",
    "switchport trunk native vlan 999",
    "switchport trunk allowed vlan"
  ]
}
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

JSON in Python

json.dumps()

Converts provided data type to string

Accepts:

- Dictionary
- List

Optional Argument:

- indent – accepts none negative Integer and pretty print the data

json.loads()

Converts provided string to valid Python data type.

Returns:

- Dictionary
- List

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Work with YAML Files

- Python uses a PyYAML module to work with YAML. It is not part of the standard module library, so it needs to be installed:
- **pip install pyyaml**
- Read from YAML

.loads = read
.dumps=to save

yaml_read.py > ...

```python
import yaml
from pprint import pprint

with open('info.yaml') as f:
    templates = yaml.safe_load(f)
"""
safe_load() only contains a subset of the load function,
all the scary arbitratry code exectution type of stuff is stripped out
"""

#print(templates)

pprint(templates)
```

```
[{'BS': 1550, 'IT': 791, 'id': 11, 'name': 'Liverpool', 'to_id': 1, 'to_name': '
LONDON'}, {'BS': 1510, 'IT': 793, 'id': 12, 'name': 'Bristol', 'to_id': 1, 'to_n
ame': 'LONDON'}, {'BS': 1650, 'IT': 892, 'id': 14, 'name': 'Coventry', 'to_id':
2, 'to_name': 'Manchester'}]
[{'BS': 1550,
  'IT': 791,
  'id': 11,
  'name': 'Liverpool',
  'to_id': 1,
  'to_name': 'LONDON'},
 {'BS': 1510,
  'IT': 793,
  'id': 12,
  'name': 'Bristol',
  'to_id': 1,
  'to_name': 'LONDON'},
 {'BS': 1650,
  'IT': 892,
  'id': 14,
  'name': 'Coventry',
  'to_id': 2,
  'to_name': 'Manchester'}]
```

Info.yaml

info.yaml

```yaml
#list of dictories
- BS: 1550
  IT: 791
  id: 11
  name: Liverpool
  to_id: 1
  to_name: LONDON
- BS: 1510
  IT: 793
  id: 12
  name: Bristol
  to_id: 1
  to_name: LONDON
- BS: 1650
  IT: 892
  id: 14
  name: Coventry
  to_id: 2
  to_name: Manchester
```

Work with YAML Files

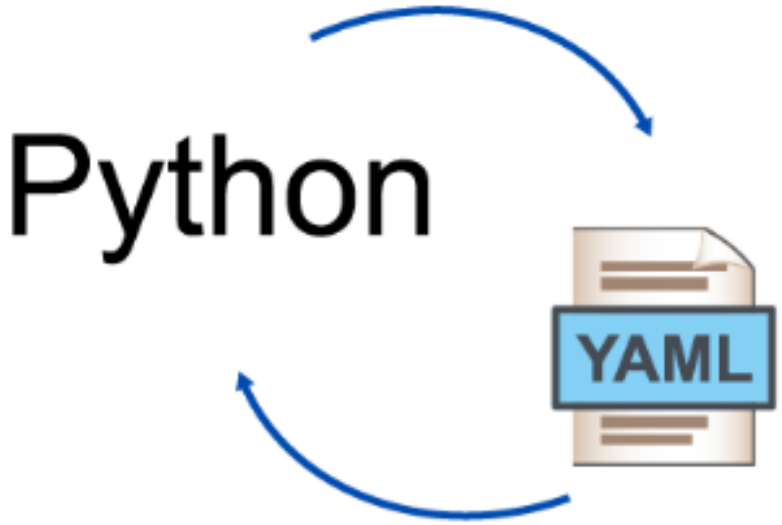
- **Writing to YAML**

yaml_write.py > ...

```python
import yaml

trunk_template = [
    'switchport trunk encapsulation dot1q', 'switchport mode trunk',
    'switchport trunk native vlan 999', 'switchport trunk allowed vlan'
]
access_template = [
    'switchport mode access', 'switchport access vlan',
    'switchport nonegotiate', 'spanning-tree portfast',
    'spanning-tree bpduguard enable'
]
to_yaml = {'trunk': trunk_template, 'access': access_template}

with open('sw_templates.yaml', 'w') as f:
    yaml.dump(to_yaml, f, default_flow_style=False)
#there are three flow styles:True, False, and None)

with open('sw_templates.yaml') as f:
    print(f.read())
```

ll rights reserved. Cisco Confidential

Convert YAML data into JSON




```
import yaml
import json
yaml_data = open('facts.yml', 'r')
data = yaml.safe_load(yaml_data)
print(json.dumps(data, indent=4))
```

```
---

hostname: nycr1
location: nyc
```

```
{

    "hostname": "nycr1",
    "location": "nyc"

}
```

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential

Lab

Lab - Parse Different Data Types with Python

Challenge-Parse the show version files

© 2020 Cisco and/or its affiliates. All rights reserved. Cisco Confidential